

Take-Home Challenge: Stateful Agentic Support & Compliance Pipeline

Objective

Design and implement a working prototype of a stateful AI workflow engine (DAG) that processes unstructured customer billing complaints. The pipeline must ingest the text, route the logic based on dynamic checks, and safely generate a validated final action.

Background Scenario

A company receives customer support emails regarding billing discrepancies. Processing these manually creates latency and compliance risks.

You need to build an automated workflow framework that handles this process safely. Instead of a rigid linear sequence of LLM calls, the application should function as a stateful graph where a unified state object is updated across execution steps, outputs are strictly validated, and execution paths branch dynamically based on the processed content.

Workflow Requirements

Your pipeline must manage a centralized application state and orchestrate a workflow that covers the following capabilities:

- **Structured Information Extraction:** Ingest raw, unstructured text and use an LLM to pull out core structured fields (such as account identifiers and claimed financial values).
- **Data Validation:** Ensure that the extracted LLM output strictly conforms to a defined structural contract. You must use data validation schemas to catch and handle missing or malformed data before it moves further into the system.
- **Internal State Verification:** Cross-reference the extracted data against a mock internal system (e.g., a local database or dictionary) to evaluate the accuracy of the claim and update the application state accordingly.
- **Compliance & Risk Check:** Evaluate the workflow context for data privacy or security violations, such as the accidental exposure of unredacted Personally Identifiable Information (PII) like credit card numbers.
- **Dynamic Routing:** The workflow execution must branch dynamically based on the results of the compliance and risk check. Safe inputs must proceed to automated response generation, while risky or flagged inputs must trigger a secure alternative path (such as logging an alert and outputting an internal escalation ticket format).

Technical Constraints

- **Language:** Python 3.10+
- **Orchestration:** You may use graph/agent orchestration frameworks or implement a custom native Python control loop to manage the state and directed execution logic.
- **State Management:** The execution must rely on a centralized, traceable global state context passed through the system.
- **Validation:** Use standard Python data validation libraries (such as Pydantic) to enforce structural boundaries.
- **Models:** You may use any accessible LLM provider or local models via tools like Ollama. Keep token efficiency and processing logic clean.

Submission Deliverables

1. **Codebase:** A clean, organized GitHub repository containing your implementation.
2. **A README.md containing:**
 - A brief overview of your system architecture, state design, and validation strategy.
 - Instructions on how to execute your test suites, including test cases for both standard execution and a compliance failure path.
 - A short comment explaining how you would optimize this pipeline for token cost and latency if scaled to high daily volumes.

Evaluation Criteria

- **State Rigor:** Clean data flow through the workflow without unintended state mutations or side-effects.
- **Validation Robustness:** How effectively your validation layer catches and handles unexpected or malformed LLM outputs.
- **Dynamic Routing Logic:** Clean implementation of conditional execution paths based on runtime evaluations.
- **Code Quality:** Readability, basic logging, and separation of concerns.